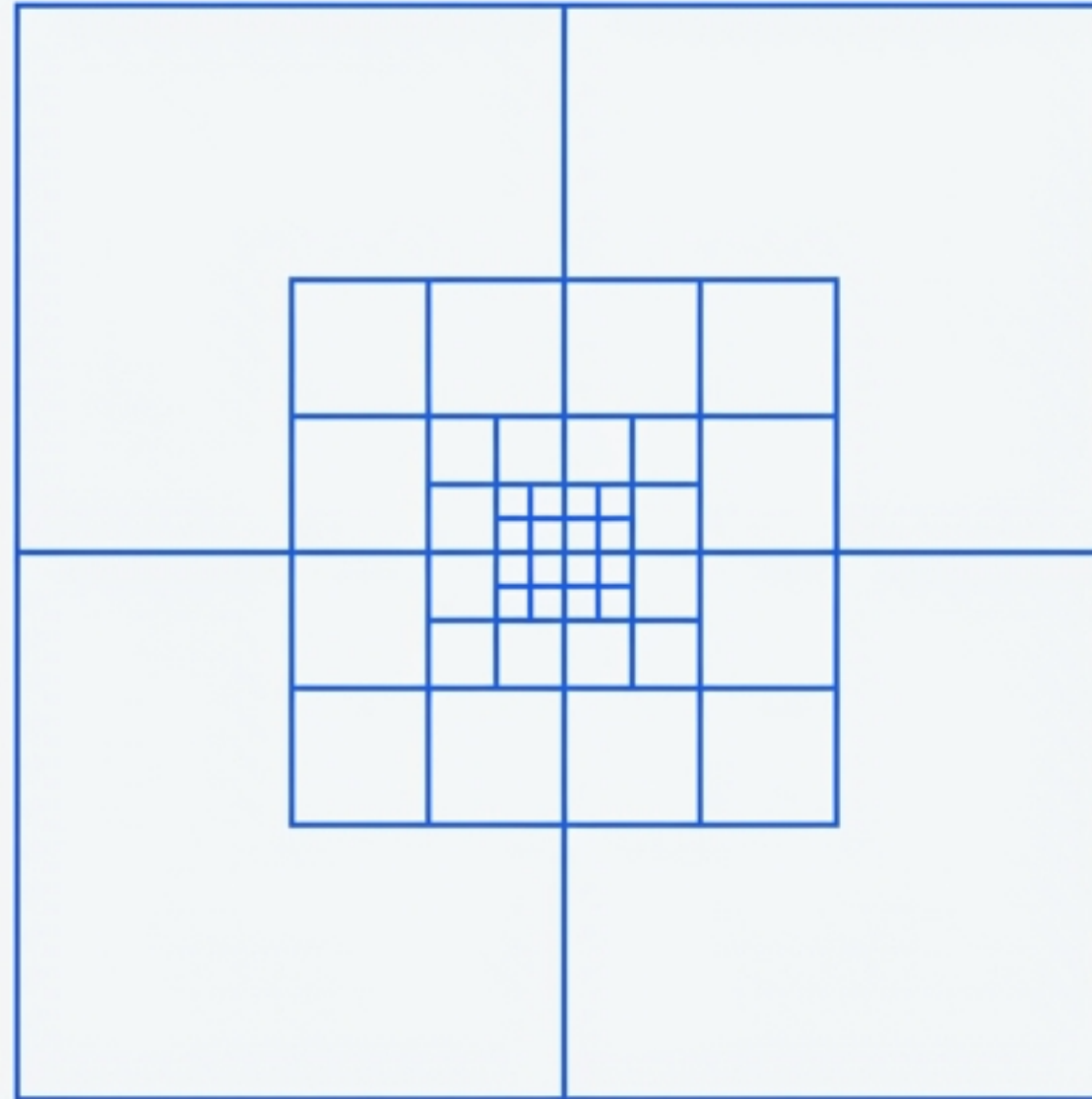


# THE RECURSIVE ISSUE MODEL



## Architecture Brief 004 — Core Principles & System Dynamics

Synthesizing briefs 001, 002, and 003 into a unified executable specification.

# Everything is an Issue.

## The Core Insight

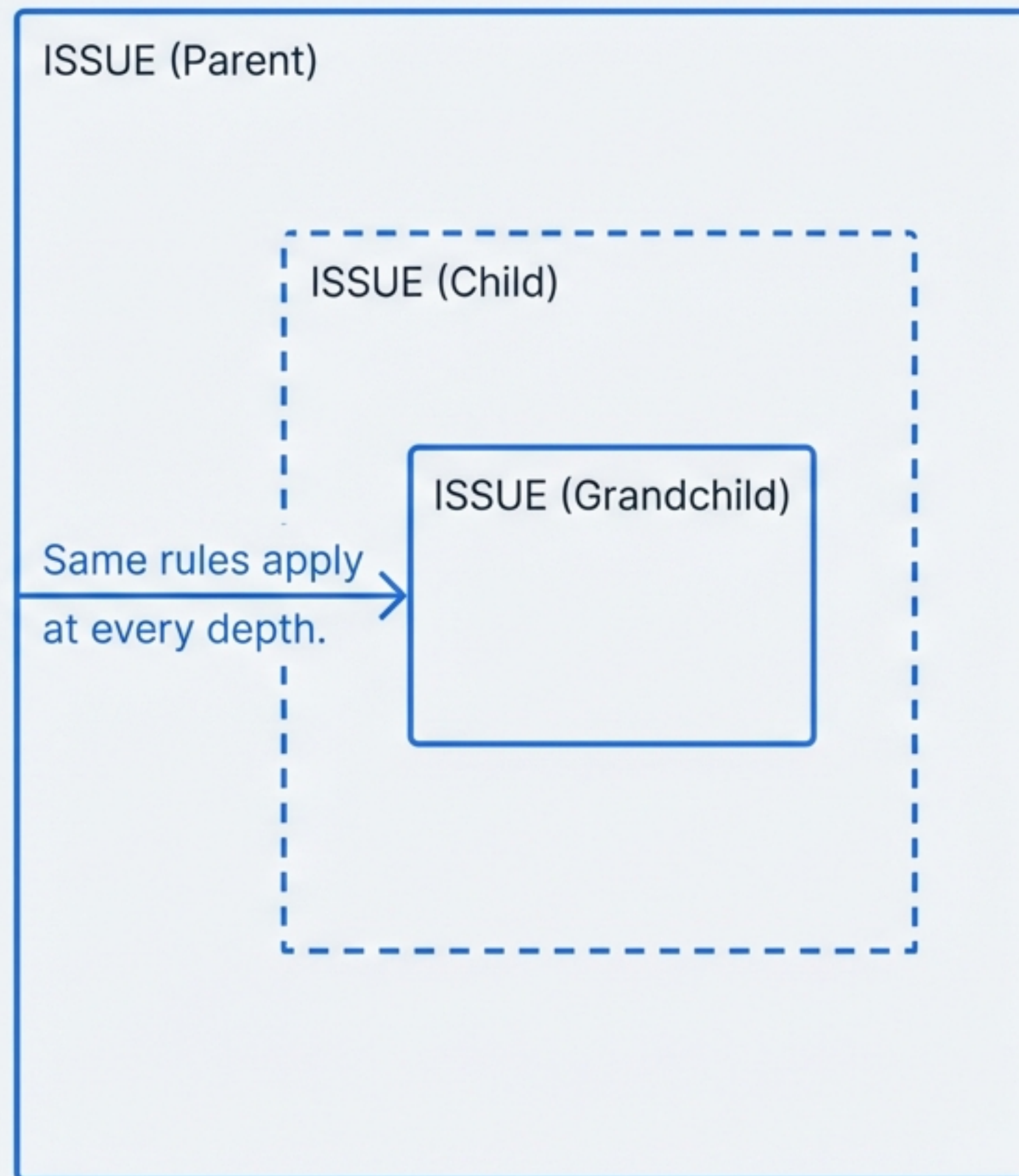
The system is built on a recursive, fractal structure. We have removed structural distinctions between “Projects,” “Epics,” and “Tasks.”

## The Paradigm

An issue contains issues.

## Implication

Any issue can be the entry point (the “root”) for a given context. The architecture supports infinite depth, limited only by utility.





# Evolution from 'Node' to 'Issue'

Abandoning legacy graph terminology for domain-specific language.



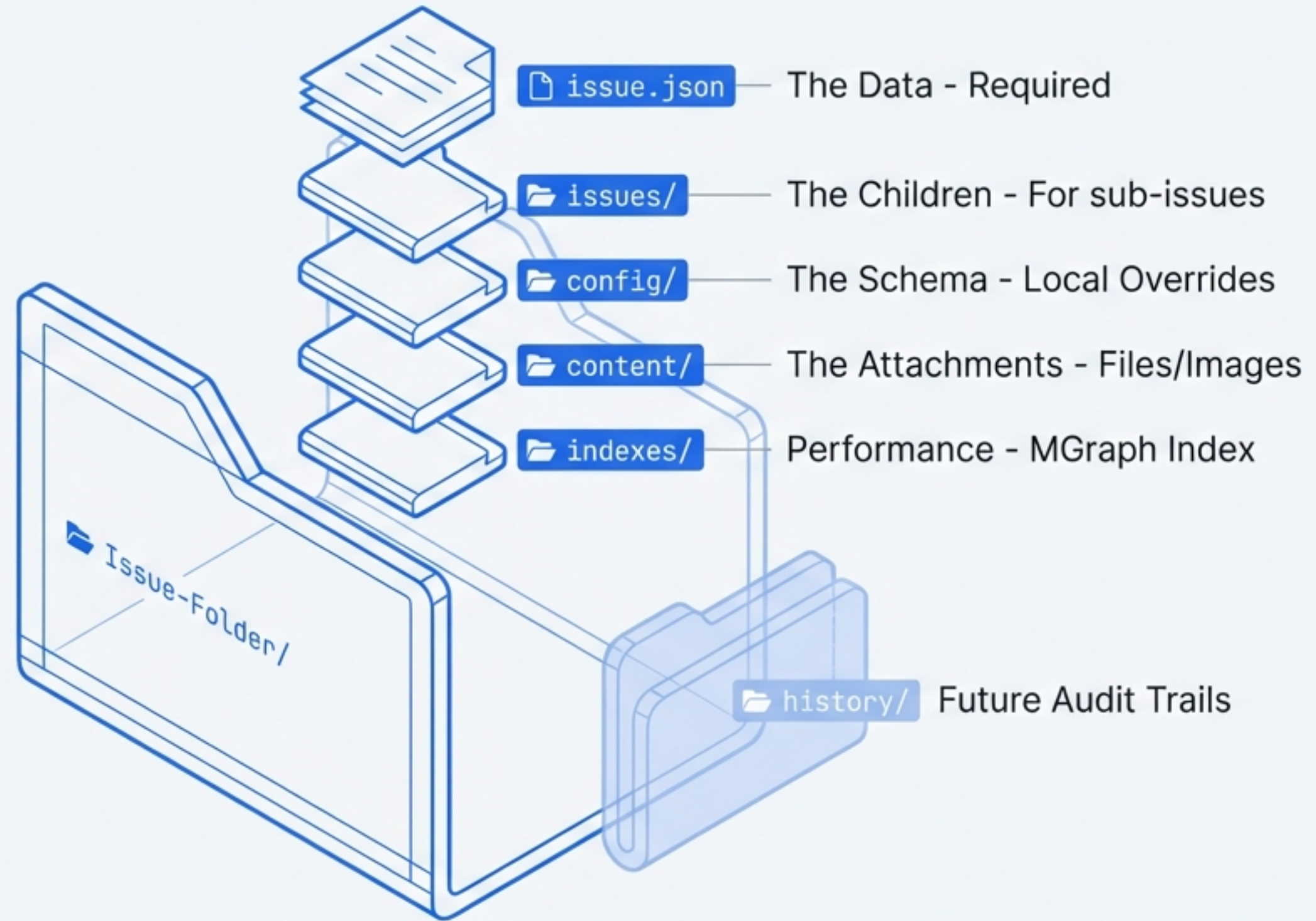
| Reason           | Explanation                                                                    |
|------------------|--------------------------------------------------------------------------------|
| Domain Language  | 'Node' is generic graph theory. 'Issue' is the natural term for our domain.    |
| Avoids Collision | 'Node' conflicts with MGraph-DB primitives. 'Issue' provides clear separation. |
| Self-Describing  | 'issue.json' immediately communicates purpose; 'node.json' is ambiguous.       |
| Consistency      | We naturally speak in terms of 'issues' in conversation and documentation.     |

**Key Takeaway:** We do not have an 'Issue' issue type. That would be redundancy. The atomic unit IS the Issue.



# Anatomy of an Issue Folder

## The Canonical Structure



Every issue is a folder. Top-level folders within it have strict semantic meaning. This standardisation ensures machine-readability without a database.



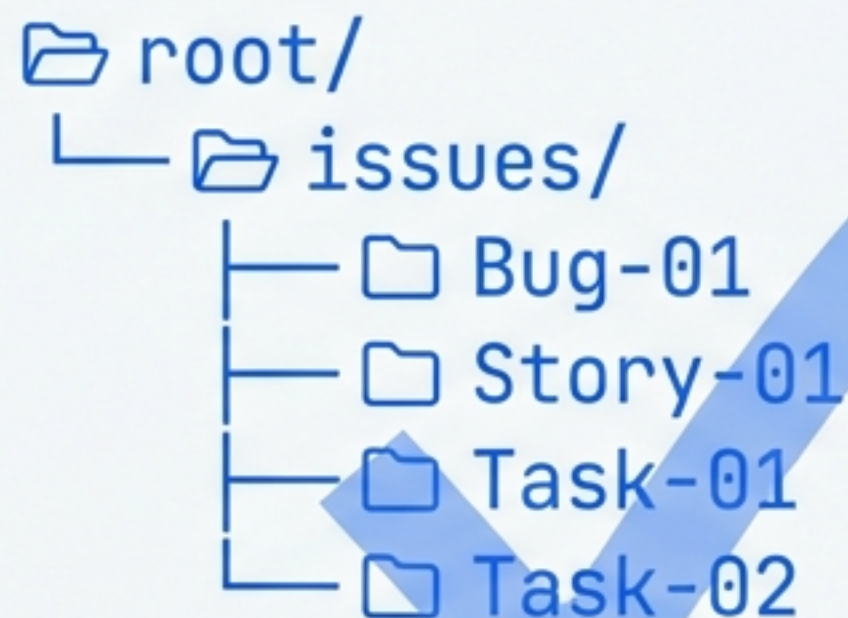
# One Folder to Rule Them All

Moving from type-segregated folders to a unified 'issues/' directory.

## OLD APPROACH



## NEW APPROACH



**Insight:** **The naming convention IS the grouping. Blueprint.**

Alphabetical sorting naturally groups types. All 'Bug-' items sit together; all 'Task-' items sit together. We do not use redundant folder structures to define types.



# What You See Is What You Get

The filesystem IS the graph.

```
$ tree issues/
├── issues
│   ├── Feature-Auth
│   │   ├── issue.json
│   │   └── issues
│   │       ├── Task-Login
│   │       └── Task-Logout
│   └── Bug-Critical
│       └── issue.json
```

No Rendering Needed: The OS file explorer provides an instant visualization.

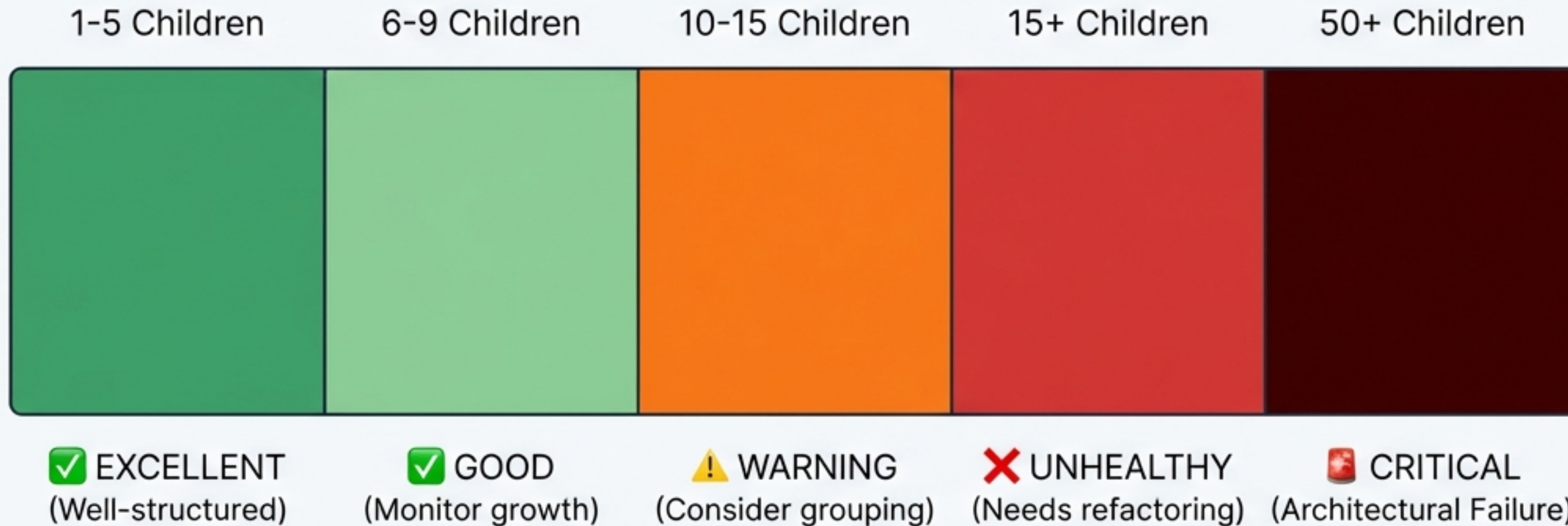
Universal Compatibility: Works with ls, GitHub, IDEs, and git status.



# The Rule of $7 \pm 2$

Using cognitive limits as architectural constraints.

## Health Meter



## Core Metric:

An issue's 'issues/' folder should ideally contain fewer than 10 direct children. High density implies a missing layer of abstraction.



# Organic Growth Pattern

Start Simple, Evolve Complexity.

START FLAT



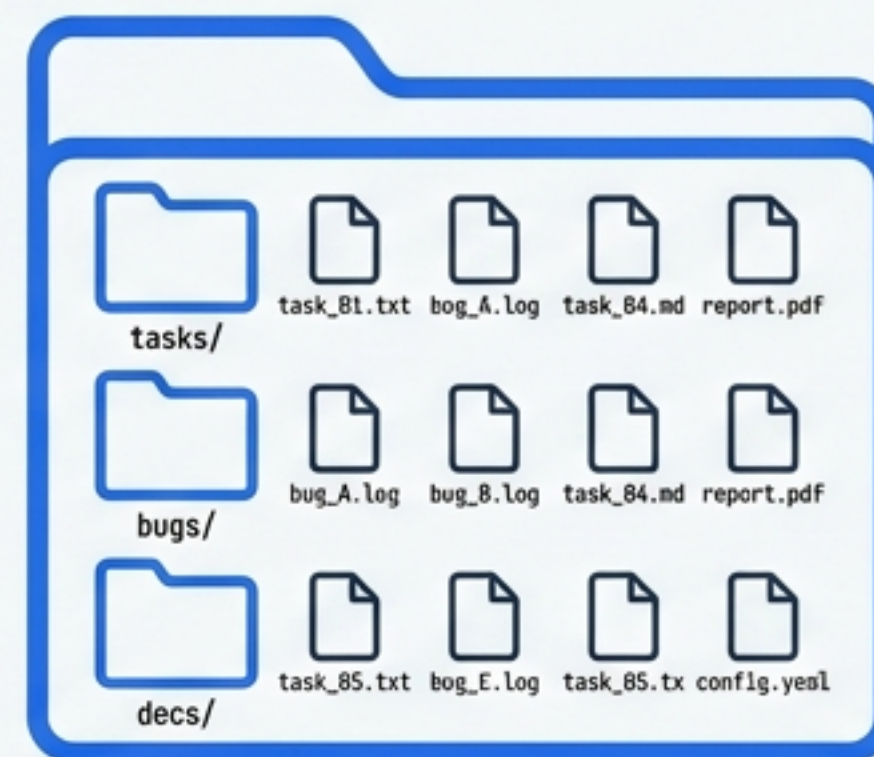
Throw items into root.

NOTICE GROWTH



Rule of  $7 \pm 2$  Triggered.

REFACTOR DOWN



Create parents. Move items. Repeat.

*No upfront design required. The structure evolves with understanding.*



# Two Modes of Identification

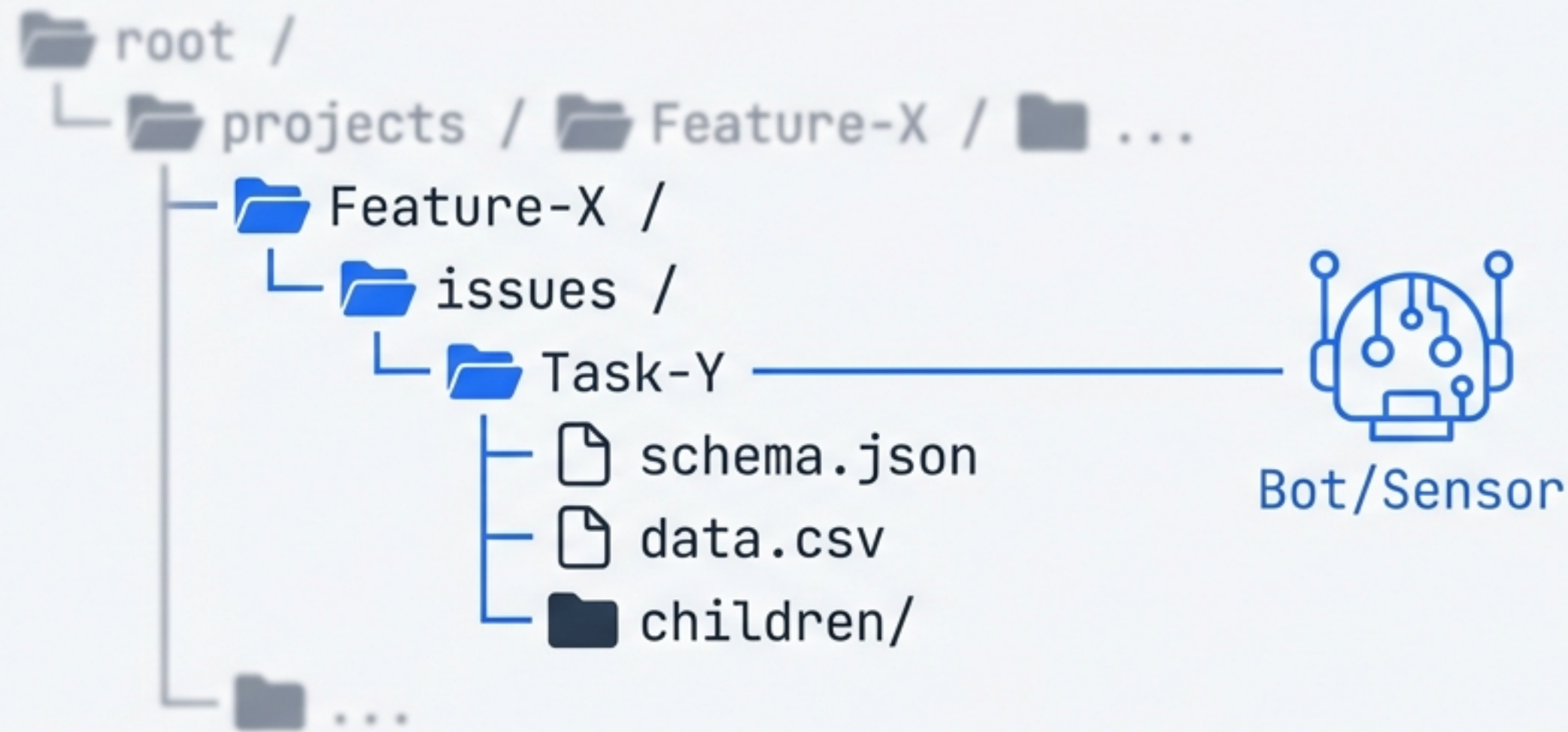
Balancing readability with precision.

| HUMAN MODE                                                                                                                                                                                                                                                                         | MACHINE MODE                                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div></div> <div>Task-Frontend-01</div> <div><b>**The Label**</b><br/><b>Purpose:</b> Human-readable, self-describing, sortable.<br/><b>Reality:</b> Collisions are expected and handled.</div> | <div></div> <div>a1b2-c3d4-e5f6-7890</div> <div><b>**The Issue ID**</b><br/><b>Purpose:</b> Machine-precision, absolute reference.<br/><b>Reality:</b> Unique across the entire graph.</div> |



# Agent Context

Self-Contained Worlds for AI.



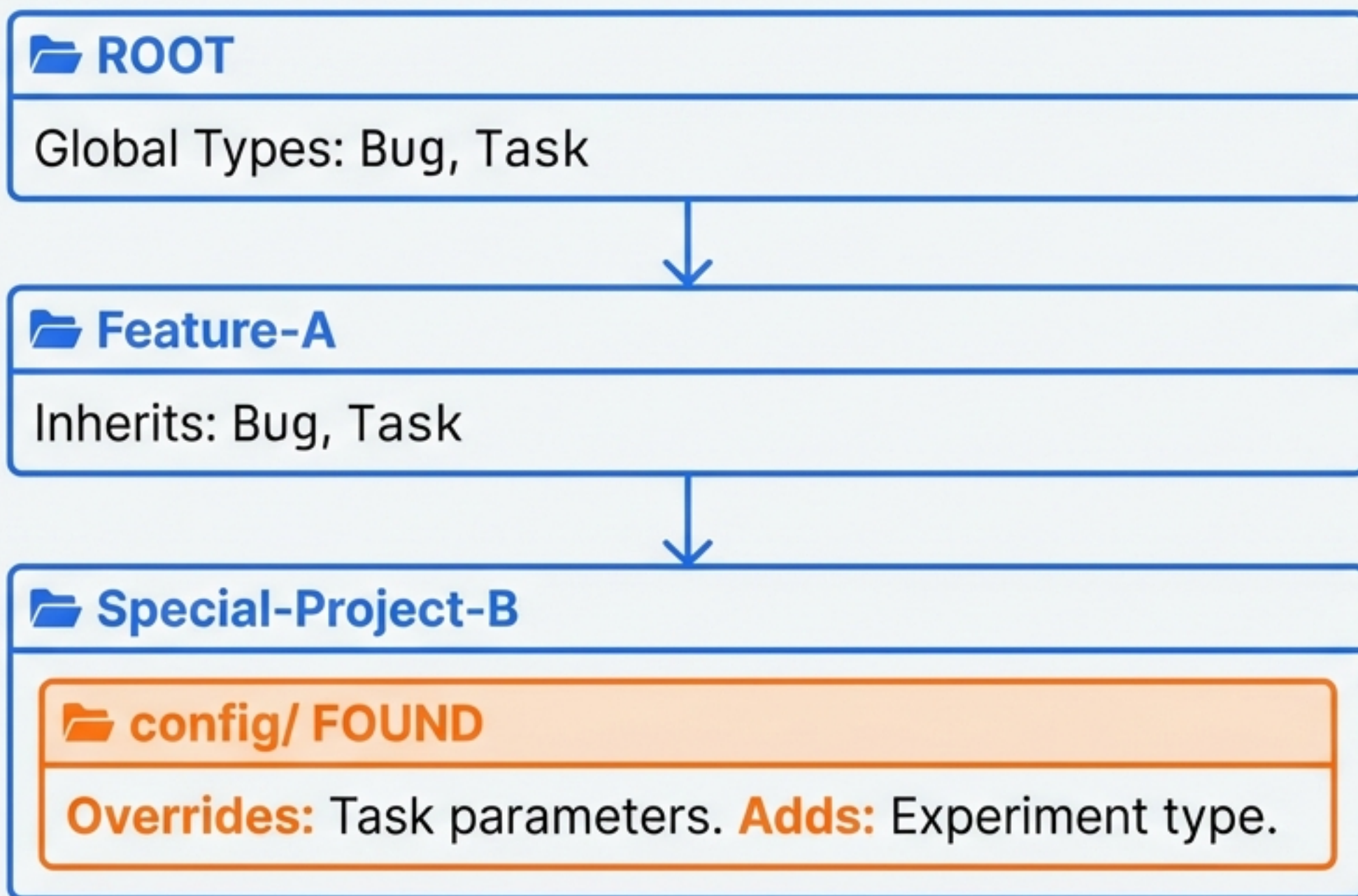
## Agent Context = Folder Path

Because every folder is a complete ecosystem (containing data, schema, and children), an AI agent can be dropped into any specific path. It operates safely within its designated bounds without needing to parse the global graph.



# Schema Inheritance

Local Definitions, Global Consistency.

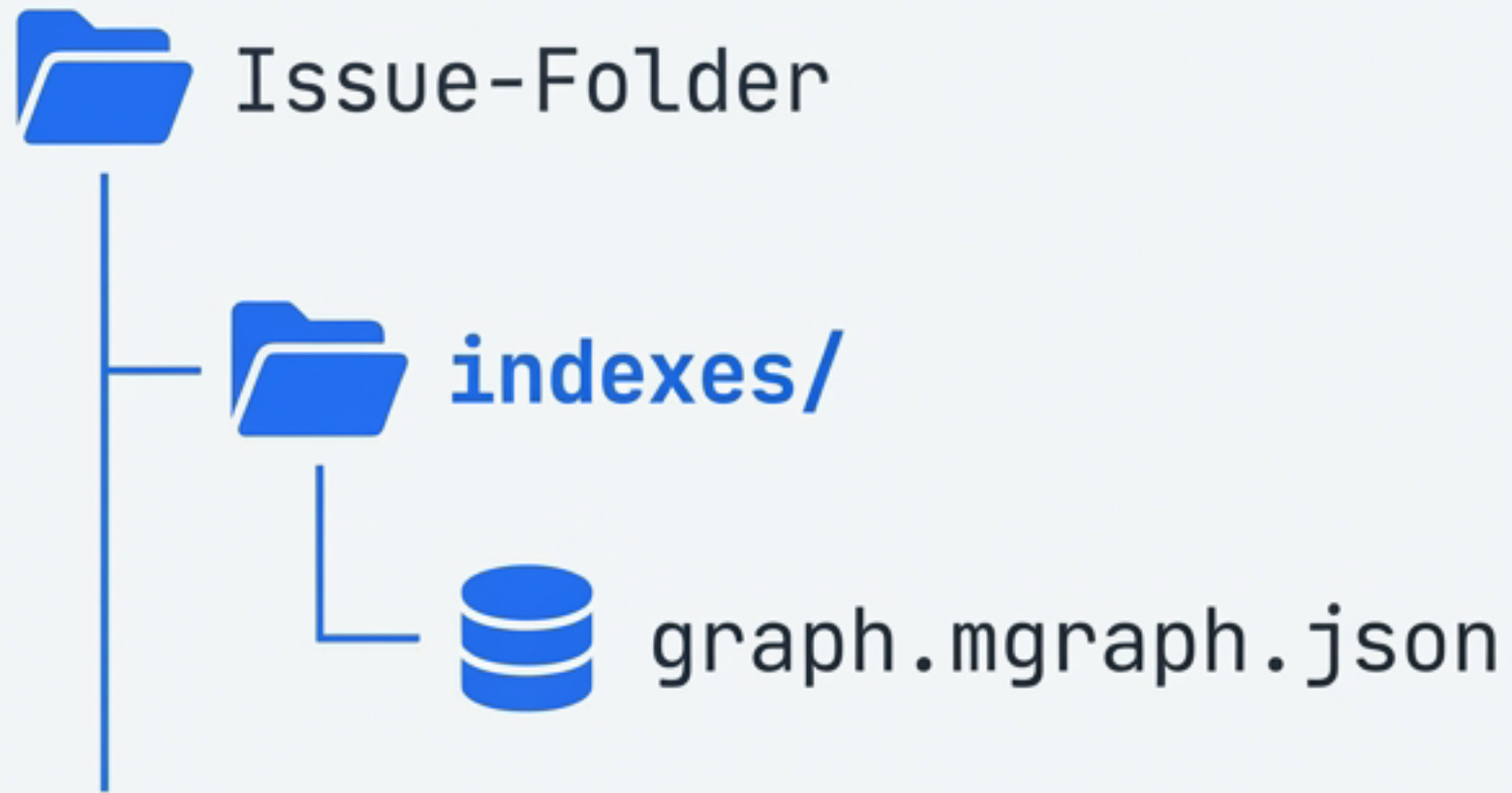
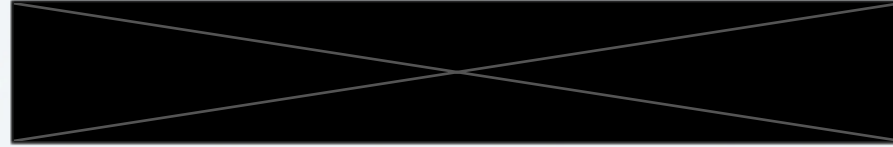


Each layer of the hierarchy can define its own schema in the 'config/' folder. These rules cascade down, overriding parent settings where specified.



# Indexing & MGraph

Optimising for Scale.



**Problem:** Traversing thousands of folders on disk is slow for complex queries.

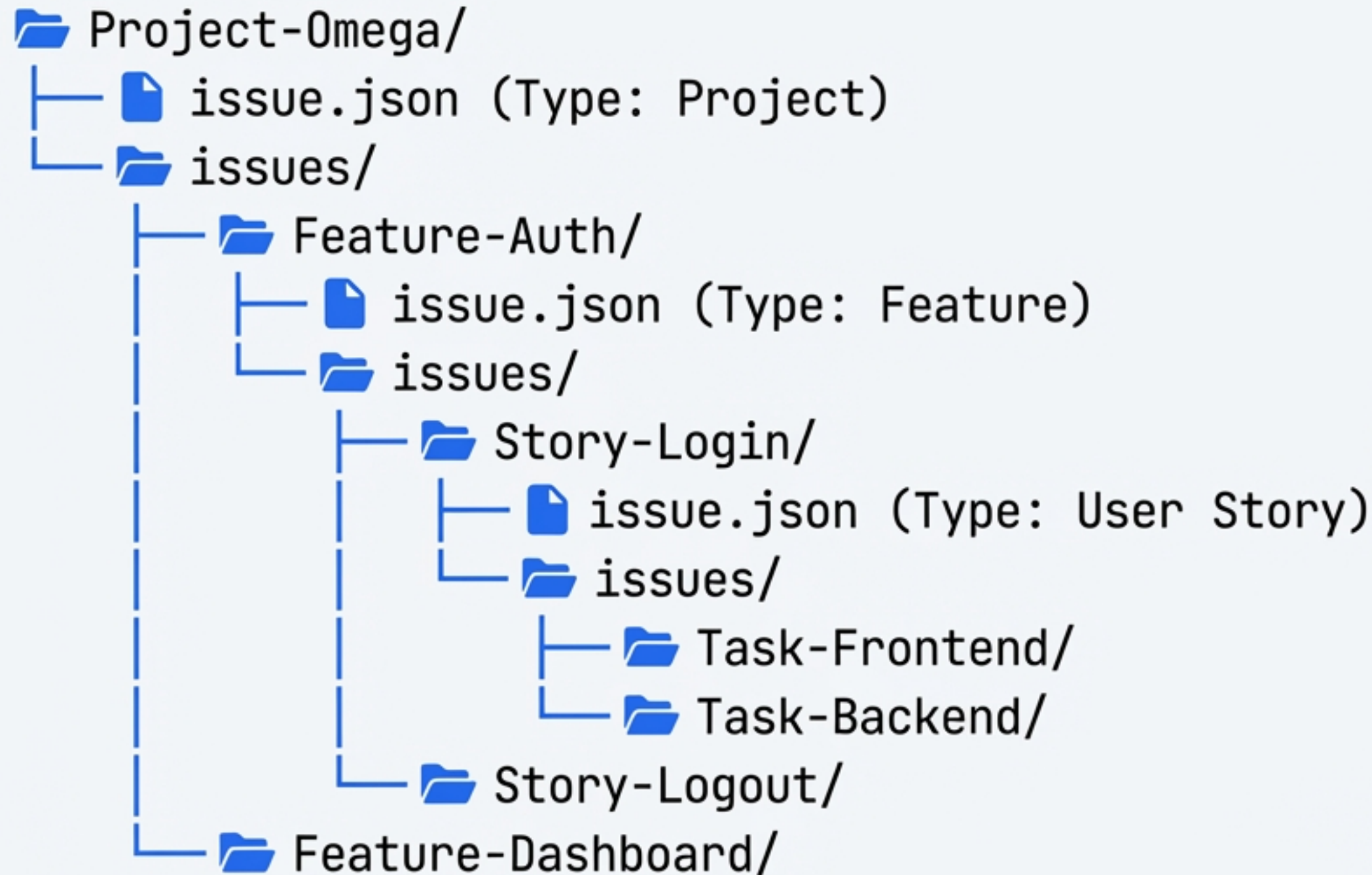
**Solution:** MGraph Index files.

**Key Principle:** The filesystem remains the Source of Truth. The MGraph index is merely a generated artifact for read-performance.



# Project Structure Example

Putting it all together.



← All of these are technically just "Issues" nested at different depths.



# The Ten Core Principles

1. Everything is an Issue.
2. Filesystem = Graph.
3. Atomic Unit is “Issue” (not Node).
4. Unified “[issues/](#)” Folder (No types).
5. Naming Convention = Grouping.
6. Rule of  $7 \pm 2$  ([Health Metric](#)).
7. Organic Growth ([Start Flat, Refactor](#)).
8. Agent Context = Folder Path.
9. Schema Inheritance.
10. Filesystem is Truth; Index is Optimization.



# ARCHITECTURE COMPLETE

System Established.

This document synthesizes all learnings from briefs 001, 002, and 003. The recursive model is now the standard for implementation.

Next Steps:  
Implementation of standard.  
Adoption of 'issue' nomenclature.

**STATUS: FINAL | DATE: 01 FEB 2026**